UWI

MONA CAMPUS
JAMAICA, WEST INDIES

Course Code: INFO3180

Course Name: Dynamic Web Development II

Group Project Documentation

Faculty of Science and Technology

Lecturer: Mr. Laurie Leitch

Submission Date: May 7, 2026

Table of Contents

1. Overview

DriftDater is a web platform that helps you to connect with people who share the same vibe as you, interests and location. It also allows you to create detailed profiles, discover compatible matches and initiate connections with other users.

Main Features include:

1. User Authentication
2. Profile Browsing
3. Matching System
4. Messaging and Connections
5. Filtering system
6. Favourite Profiles

Additional Features include:

1. Notification System
2. Report and Block User Functionality
3. Dark Mode Customization

The Frontend is built using:

- Vue.js
- Vue Router
- Responsive CSS Layouts
- API Services

The Backend consists of handling:

- User Authentication
- API routing
- Database Communication

2. Challenges and Solutions

2.1 Frontend and Backend Integration

One of the main challenges was getting the Vue frontend and Flask backend to communicate properly, especially when testing API requests and handling user sessions. This was resolved by fixing API routes, configuring CORS correctly, and organizing the API calls more consistently across the frontend.

2.2 Block and Report Button Functionality

During frontend testing, the block and report buttons were initially not clickable because of event handling and component structure issues within the browse cards. This was resolved by adjusting the button event listeners and reorganizing the modal popup structure so that the buttons responded correctly to user interaction.

2.3 Blocked User Filtering

After implementing the block functionality, blocked users were still appearing in the browse page results. This issue occurred because the browse query was not filtering blocked relationships from the database. The problem was resolved by updating the browse query to exclude users who were blocked by the current user, as well as users who had blocked the current user.

2.4 User Interface Improvements

Initially, the report feature displayed the report form directly on the browse page, which made the page feel cluttered and difficult to use. To improve the user experience, popup modals were added for both the block and report features, making the interface cleaner and easier to interact with.

2.5 Dark Mode Styling

Implementing dark mode required additional CSS overrides because several page components and containers were still using the default light theme styling. This was fixed by adding global dark mode styles and stronger CSS overrides to ensure the theme applied consistently across the application.

2.6 Favourite System Integration

While implementing the favourite profiles feature, there were issues with the frontend and backend routes not matching correctly. Some API requests were being sent to outdated routes, which caused favourite profiles to fail to save or load properly. This was resolved by updating the frontend API calls to match the backend routes and restructuring the favourites page so saved profiles could be displayed and removed correctly.

2.7 Matching System and Notification Synchronization

During development of the matching system, mutual matches were being created successfully in the database, but match notifications were not appearing in the notifications page. This occurred because notification records were not being generated when a mutual match was detected. The issue was resolved by adding automatic notification creation within the matching logic whenever two users liked each other successfully.

Future Improvement

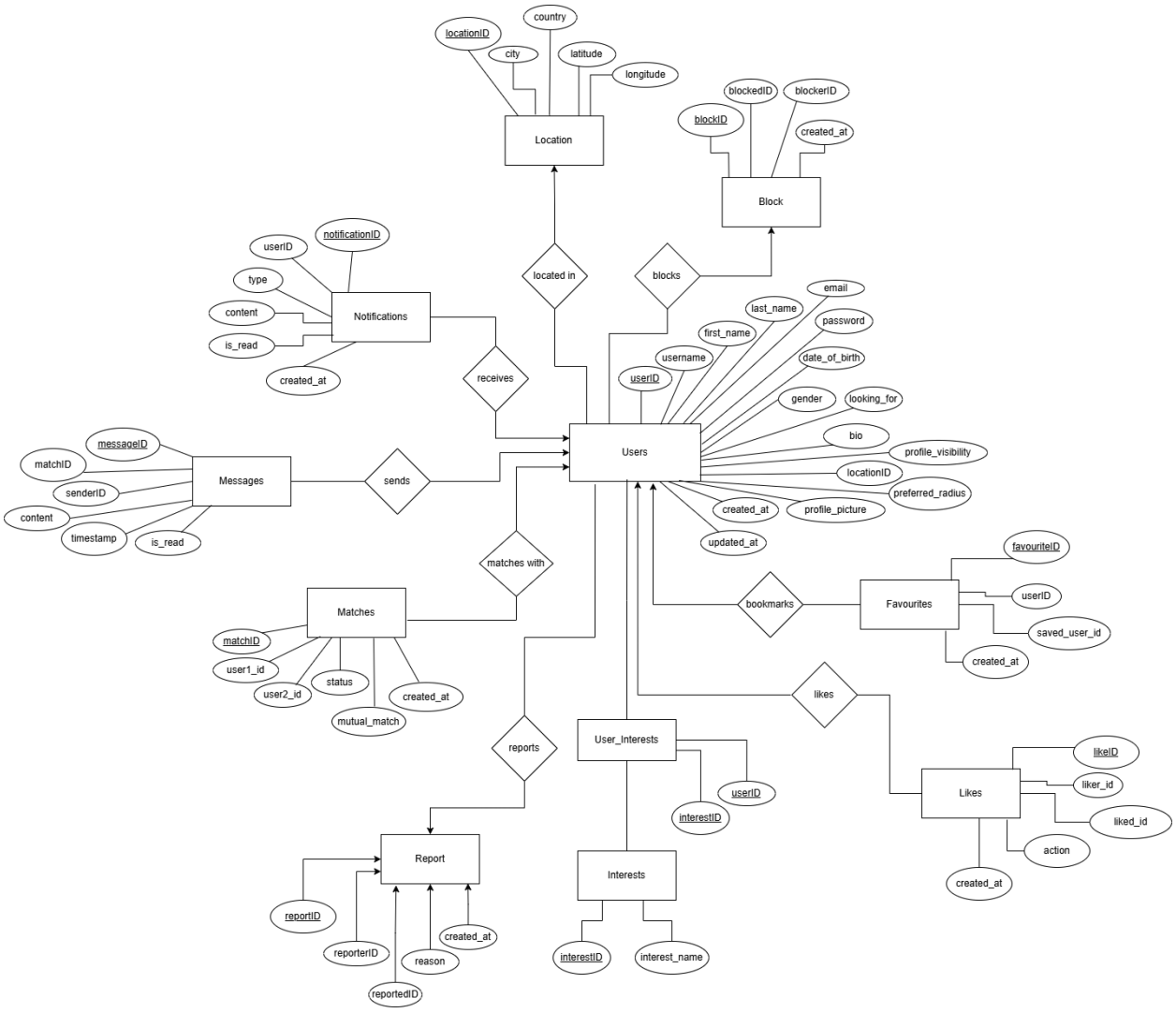
3.1 Have a separate profile view and profile editing page

A future improvement for DriftDater would be to separate profile viewing from profile editing. Currently, the profile page is mainly used for editing profile information. A more polished version of the application would first display a public facing profile page (think Instagram profile page) showing the user's profile picture, bio, interests, and other profile details, with a separate Edit Profile button that opens the profile editing form.

3.2 Have a user profile page

Another future improvement would be to allow users to click profile cards from the Browse page to open a dedicated profile viewing page. This page could include features such as liking profiles and viewing additional information (like a users pictures)

4. Entity-Relationship Diagram



5. Database Schema

Users(userID, username, first_name, last_name, email, password, date_of_birth, gender, looking_for, bio, profile_visibility (public/private), locationID, preferred_radius, profile_picture, created_at, updated_at)

Location(locationID, city, country, latitude, longitude)

Interests(interestID, interest_name)

User_Interests(userID, interestID)

Matches(matchID, user1_id, user2_id, status (liked/passed/matched), mutual_match, created_at)

Messages(messageID, matchID, senderID, content, timestamp, is_read)

Favourites(favouriteID, userID, saved_user_id, created_at)

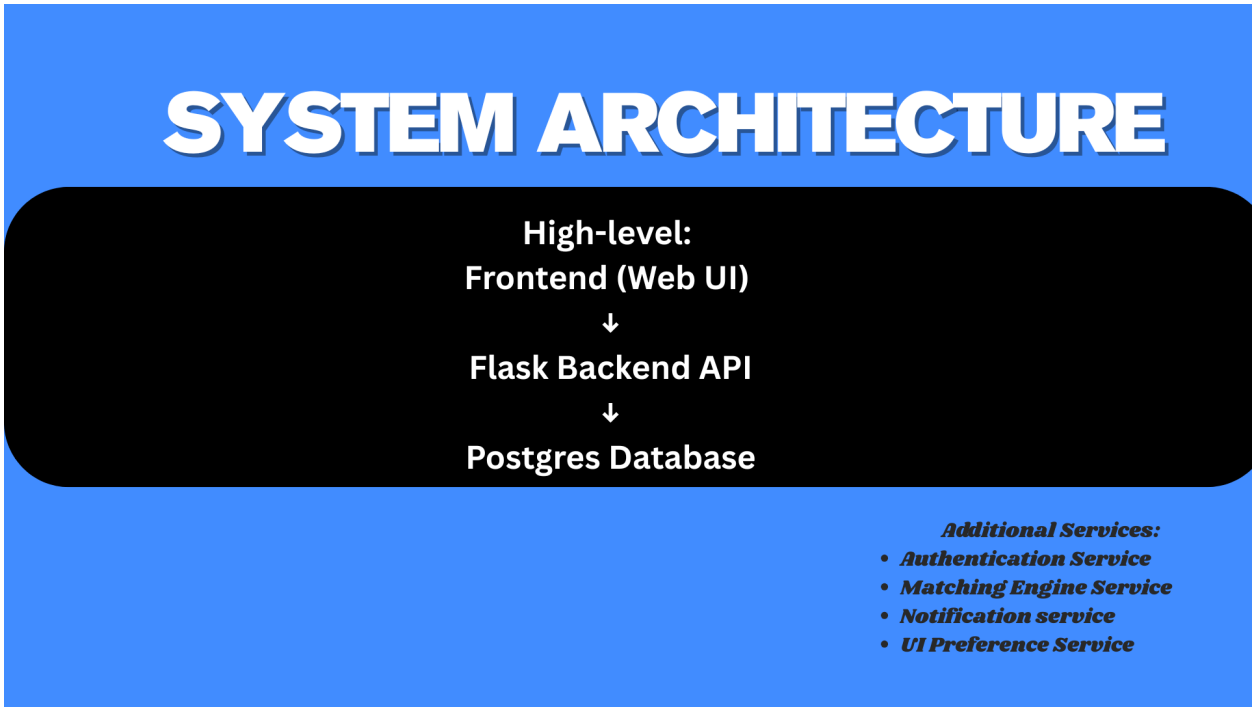
Notifications(notificationID, userID, type (match, message, like), content, is_read, created_at)

Block(blockID, blockerID, blockedID, created_at)

Report(reportID, reporterID, reportedID, reason, created_at)

Likes(likeID, liker_id, liked_id, action (like/pass), created_at)

6. System Architecture



7. Team Roles & Contributions

Team Members	ID Numbers	Roles	Contributions
Marissa O'Meally	620163584	Project Owner & Frontend Developer	• Created the E-R Diagram, database schema and system architecture • Implemented the search and discovery functionality and the models.py • Implemented frontends components such as: Home.vue, FavouritesView.vue and BrowseMatchesView.vue
Seantay Johnson	620155318	Backend and Frontend Developer	• Implemented messaging and conversation retrieval functionality • Implemented the Chat.vue frontend component
Gabriel Smith	620162866	Backend and Frontend Developer	Additional Features: • Notification System • Report User Functionalities • Block User Functionalities • Dark Mode Customization Troubleshooting and Fixes: • Match functionality • Favourite functionality • Location Feature
Kevon Haughton	620149400	Backend Developer	• Matching system

		and Frontend Developer	Functionalities ● Like/Pass Functionalities ● Implemented MatchesView.vue frontend component.
Deshawn Matthews	620160983	Backend and Frontend Developer	● User Authentication ● Profile Management ● Account creation ● Frontend Implemention : ProfileView RegisterVIew